ARRAYS

LEARNING OBJECTIVES

At the end of this lesson, you should be able to:

- Discuss the concept of arrays and their role in storing collections of data.
- Explain how indexing and accessing elements work in arrays.
- Demonstrate knowledge of common array operations and their practical applications.

INTRODUCTION

Arrays are a fundamental data structure in Java used to store multiple values of the same data type under a single variable name.

Arrays offer a convenient way to manage and manipulate collections of data.



DECLARING AND INITIALIZING ARRAYS

Arrays are declared using the syntax:

dataType[ ] arrayName;

Arrays can be initialized during declaration or later.

Example:

```
// Declare an integer array
int[] numbers;

// Declare and initialize a double array
double[] grades = {90.5, 85.0, 78.2};
```

INDEXING AND ACCESSING ELEMENTS

Indexing is a fundamental concept in arrays that allows us to access specific elements within an array.

Each element in an array is assigned a unique index, starting from 0 for the first element.



ZERO-BASED INDEXING

Java uses zero-based indexing, meaning the first element is at index 0, the second at index 1, and so on.

Example:

```
int[] numbers = {10, 20, 30, 40, 50};
// Index: 0  1  2  3  4
```



ACCESSING ELEMENTS

Elements within an array can be accessed using their index within square brackets.

Example:

```
// Accessing the third element (30)
int thirdNumber = numbers[2];
```

INDEXING OUT OF BOUNDS

Indexing out of bounds: Trying to access an element at an index that doesn't exist in the array.

Example:

```
// Erro: Index 5 is out of bounds
int invalidNumber = numbers[5];
```



USING LOOPS FOR ACCESS

Loops are often used to iterate through arrays and access each element.

This is useful when you want to perform a specific operation on all elements.

Example:

```
for (int i = 0; i < number.length; i++) {
    System.out.println(array[i]);
}
```

EXAMPLE: STORING STUDENT GRADES

Suppose you have a class of students and you want to store their exam grades.

```
public class StudentGrades {
    public static void main(String[] args) {
        double[] examGrades = {95.5, 88.0, 76.5, 92.3, 81.8};

        System.out.printIn("Student Exam Grades:");
        for (int i = 0; i < examGrades.length; i++) {
            System.out.printIn("Student " + (i + 1) + ": " + examGrades[i]);
        }
    }
}
```

The loop iterates through the array, printing each student's grade along with their corresponding index.

In this example, an array examGrades is used to store the exam grades of five students.

MID-LESSON QUESTION

1. Write a Java program to find the largest and smallest elements in an array of integers.



ANSWER :MID-LESSON QUESTION

```java
public class MinMaxFinder {
    public static void main(String[] args) {
        int[] numbers = {45, 18, 73, 27, 92, 15, 64}; // Change the array here

        int smallest = numbers[0];
        int largest = numbers[0];

        for (int i = 1; i < numbers.length; i++) {
            if (numbers[i] < smallest) {
                smallest = numbers[i];
            }
            if (numbers[i] > largest) {
                largest = numbers[i];
            }
        }

        System.out.println("Smallest element: " + smallest);
        System.out.println("Largest element: " + largest);
    }
}
```

COMMON ARRAY OPERATIONS

Arrays in Java provide a range of operations to manipulate and process data efficiently.

Understanding these operations is crucial for effective use of arrays in programming.



COMMON ARRAY OPERATIONS: INSERTION

Arrays have a fixed size, so inserting elements involves shifting other elements.

Example:

```
int[] numbers = {10, 20, 40, 50};
// Insert 30 at index 2
for (int i = numbers.length - 1; i > 2; i--) {
    numbers[i] = numbers[i - 1];
}
```

COMMON ARRAY OPERATIONS: DELETION

Arrays have a fixed size, so deleting elements involves shifting other elements.

Example (Deletion):

```
int[] values = {10, 20, 30, 40, 50};
// Delete element at index 2
for (int i = 2; i < values.length - 1; i++) {
    values[i] = values[i + 1];
}
```

COMMON ARRAY OPERATIONS: UPDATING AND MODIFYING

Elements in an array can be updated by assigning new values.

Example:

```
int[] quantities = {5, 10, 15};
quantities[1] = 12; // Update the second element to 12
```

COMMON ARRAY OPERATIONS: SEARCHING

Linear Search: Traverse the array to find a specific element.

Binary Search: Requires a sorted array, efficiently finds elements.

Example (Linear Search):

```
int[] ages = {25, 30, 22, 18, 40};
int target = 22;
int index = -1;
for (int i = 0; i < ages.length; i++) {
    if (ages[i] == target) {
        index = i;
        break;
    }
}
```

COMMON ARRAY OPERATIONS: SORTING

Arrays can be sorted in ascending or descending order using built-in sorting algorithms.

Example:

```
int[] unsorted = {45, 10, 78, 23, 56};
Arrays.sort(unsorted); // Sort in ascending order
```

ITERATING THROUGH ARRAYS

Loops are used to traverse arrays and perform operations on each element.

Example:

```
String[] names = {"Alice", "Bob", "Charlie"};
for (String name : names) {
    System.out.println(name);
}
```

FINDING MAXIMUM AND MINIMUM

Iterate through an array to find the maximum and minimum elements.

Example:

```
int[] values = {34, 78, 12, 56, 90};
int max = values[0];
int min = values[0];
for (int value : values) {
    if (value > max) {
        max = value;
    }
    if (value < min) {
        min = value;
    }
}
```

AGGREGATE OPERATIONS

Perform aggregate operations on array elements, such as summing or averaging.

Example (Sum):

```
int[] quantities = {5, 10, 15, 8, 12};
int sum = 0;
for (int quantity : quantities) {
    sum += quantity;
}
```

ADVANTAGES OF USING ARRAYS

Grouping Data: Arrays allow us to group related data together, making it easier to manage and work with the data.

Efficient Access: Elements in an array can be accessed directly using their index, making data retrieval efficient.

Consistency: Arrays enforce a consistent data type for all elements, reducing data type-related errors.

Iterative Operations: Arrays work well with loops, enabling efficient iteration and processing of data.

Summary

SUMMARY

- Arrays are essential for storing collections of data in Java.
- They allow us to group related data and efficiently access elements using indices.
- Arrays are commonly used in various programming scenarios, from simple data storage to complex algorithms.
- Indexing is essential for accessing specific elements within an array.
- Java uses zero-based indexing, with the first element at index 0.
- Loops are commonly used to iterate through arrays and access their elements.
- Common array operations enable efficient manipulation and processing of data stored in arrays.

SUMMARY

- These operations are essential for a wide range of programming tasks.
- A solid understanding of these operations enhances your ability to work with arrays effectively.

FURTHER READING RESOURCES

- Sierra, K., & Bates, B. (2005). Head First Java: A Brain-Friendly Guide. " O'Reilly Media, Inc.".
- Horstmann, C. S. (2000). Core JAVA Vol Fundamentals.
- Burd, B. (2017). Beginning programming with Java for Dummies. John Wiley & Sons.
- Eckel, B. (2003). Thinking in JAVA. Prentice Hall Professional.
- Cosmina, I. (2018). Java for Absolute Beginners: Learn to Program the Fundamentals the Java 9+ Way. Apress.
- Schildt, H. (2003). Java™ 2: A Beginner's Guide

Thank You